

Chapter 7. Operators and Custom Resource Definitions (CRDs)

Kubernetes comes with a core feature set to fulfill the basic needs for running application stacks with a standard set of primitives. For custom use cases, Kubernetes allows for installing extensions to the platform, *operators*.

A Custom Resource Definition (CRD) is a Kubernetes extension mechanism (often bundled with an operator) for introducing custom API primitives to fulfill requirements not covered by built-in primitives.

This chapter will focus on the installation and configuration of operators, as well the interaction with provided CRDs.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Understand CRDs, install and configure operators
-

Working with Operators

Operators extend the core behavior of the Kubernetes cluster without actually changing the Kubernetes code. You can think of the operator as a plug-in to the platform. Operators typically automate tasks that would have to be performed by humans, such as deploying, configuring, scaling, upgrading, and managing applications.

The Operator Pattern

An operator usually consists of multiple components: one or more CRDs, a controller, and often additional components like RBAC rules for permissions. CRDs can be understood as the schema that defines the blueprint for a custom object, and the instantiation of those objects with the newly introduced type, also called the Custom Resources (CR).

For a CRD to be useful, it has to be backed by a controller. Controllers interact with the Kubernetes API and implement the reconciliation logic

that interacts with CRD objects.

The combination of CRDs and controllers is commonly referred to as the *operator pattern*. The exam does not require you to have an understanding of controllers; therefore, their implementation won't be covered in this chapter.

[Figure 7-1](#) shows the operator pattern with all its moving parts.

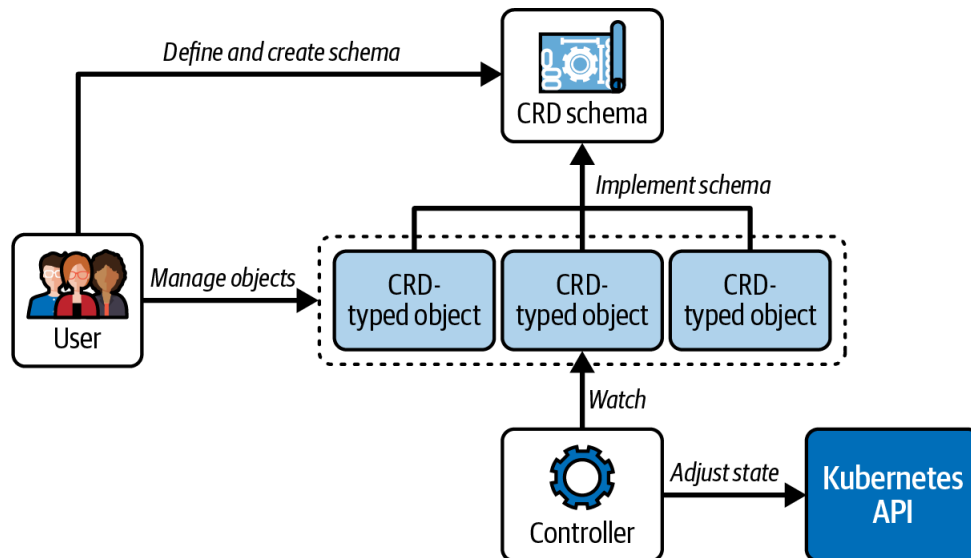


Figure 7-1. The Kubernetes operator pattern

Discovering Operators

The Kubernetes community has implemented many useful operators discoverable on [OperatorHub.io](#) or [Artifact Hub](#).

A prominent operator is the [External Secrets Operator](#) that helps with integrating external Secret managers, like Amazon Web Services (AWS) Secrets Manager and HashiCorp Vault, with Kubernetes. Another is the [Crossplane operator](#), which helps with creating and managing cloud resources using declarative syntax.

To demonstrate the functionality of OperatorHub.io, we are going to search and install the popular [Argo CD Operator](#), a declarative GitOps continuous delivery tool for Kubernetes that automates deployments by continuously monitoring applications and synchronizing them with the desired state defined in Git repositories.

In your browser, open the URL for [OperatorHub.io](#) and enter the term *argo cd* into the search box named Search OperatorHub. You will receive a result, shown in [Figure 7-2](#).

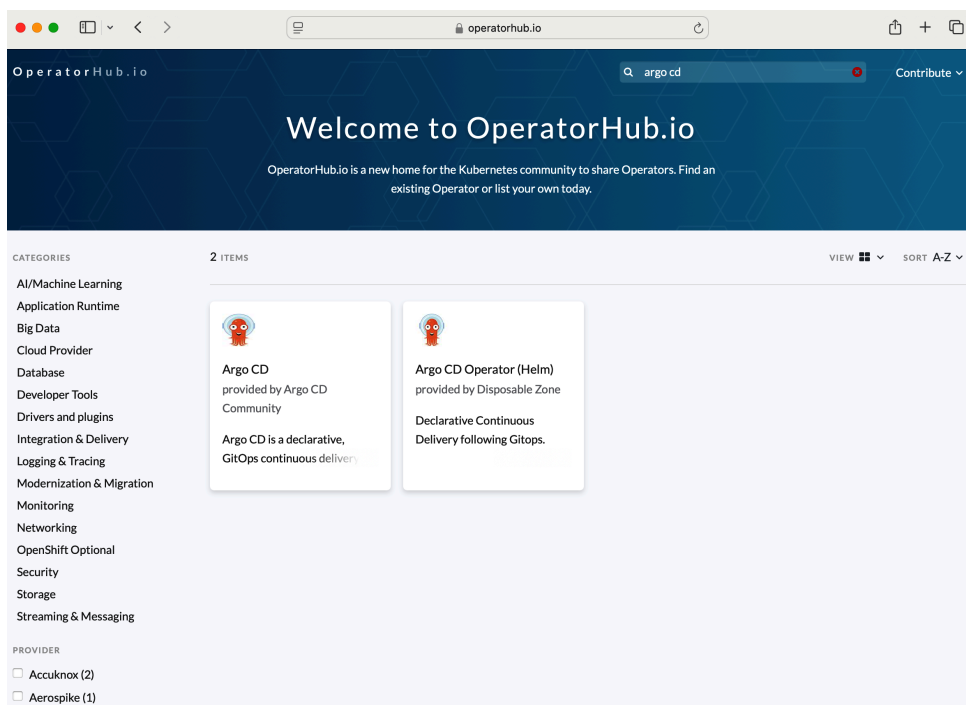


Figure 7-2. A search result on OperatorHub.io

Clicking on the Argo CD panel will bring you to the details of the [Argo CD Operator](#), shown in [Figure 7-3](#).

The page describes the functionality of the operator including a high-level overview on the provided CRDs. To be able to create CRs from the CRDs, we'll first need to install the operator.

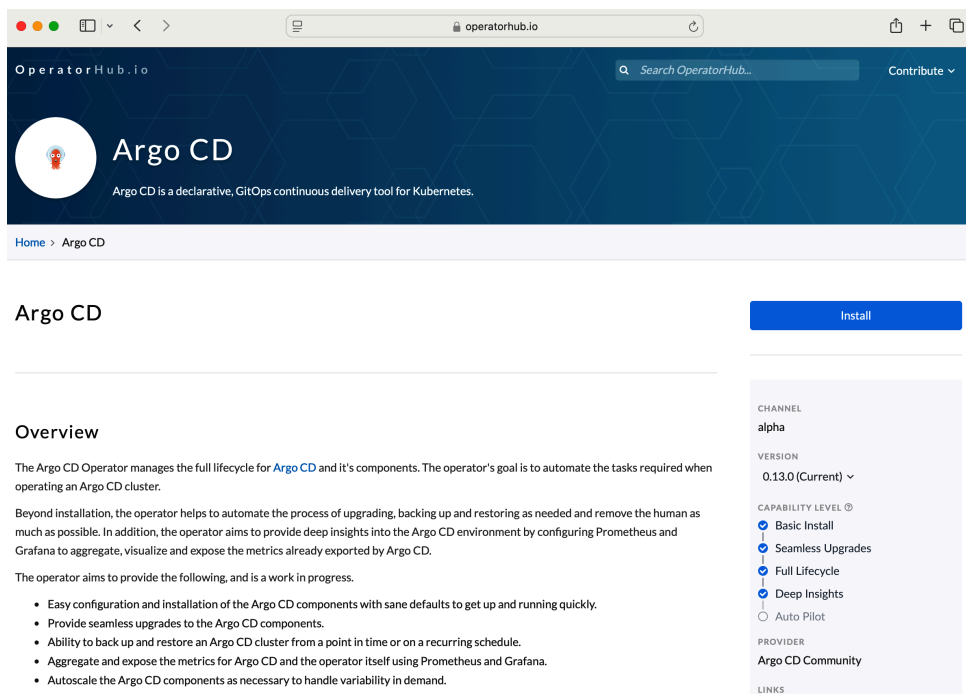


Figure 7-3. The Argo CD Operator on OperatorHub.io

Installing Operators

You can install many of those operators with a single `kubectl` command execution or by using the Helm executable. For more information on using Helm, see [Chapter 8](#).

Clicking the Install button on OperatorHub.io brings up the installation instructions, as shown in [Figure 7-4](#).

As shown on the installation page for the operator, we'll use the [Operator Lifecycle Manager \(OLM\)](#), a tool to help manage the operators running on your cluster. This is a one-time operation:

```
$ curl -sL https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.31.0/install.sh | bash -s v0.31.0
```

Next, we'll install the Argo CD Operator, which will place the operator objects into the `operators` namespace:

```
$ kubectl create -f https://operatorhub.io/install/argocd-operator.yaml
subscription.operators.coreos.com/my-argocd-operator created
```

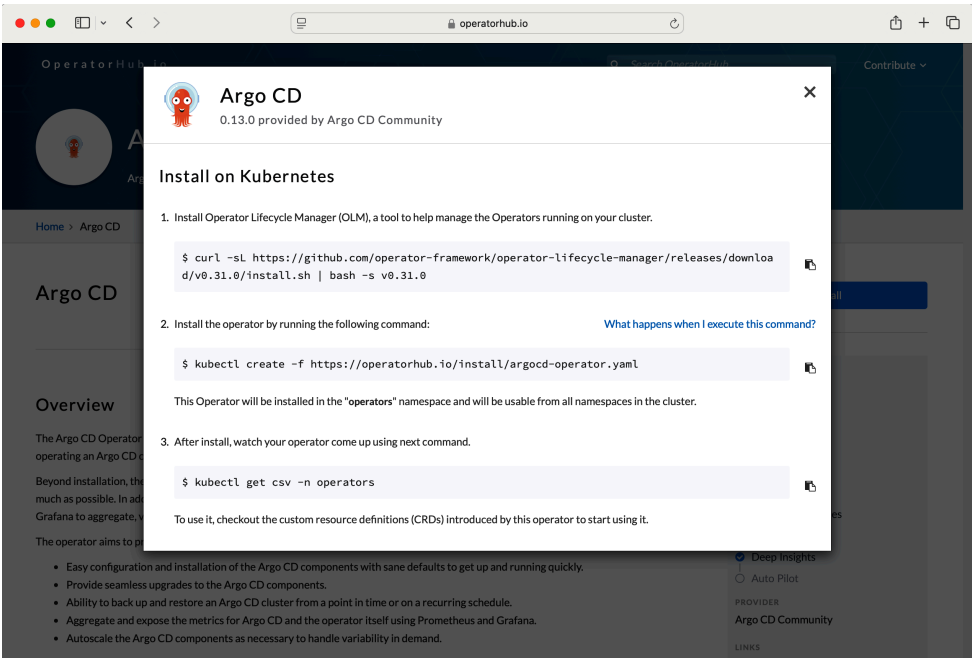


Figure 7-4. The Argo CD Operator installation instructions

You can follow the installation process by running the following command. A valid installation finishes with the `Succeeded` phase in the rendered output of the command:

```
$ kubectl get csv -n operators
```

NAME	DISPLAY	VERSION	REPLACES	PHASE
argocd-operator.v0.13.0	Argo CD	0.13.0	argocd-operator.v0.12.0	Succeeded

You are now ready to use the operator. Please refer to the next sections in the chapter to interact with the installed CRDs.

Working with Custom Resource Definitions

For the exam, you will need to understand how to discover CRD schemas provided by external operators and how to interact with objects that follow the CRD schema.

Discovering CRDs

The Argo CD Operator provides a couple of CRDs like `Application`, `ApplicationSet`, `AppProject`, and some more. The high-level purpose of each is as follows:

Application

An `Application` is a group of Kubernetes resources as defined by a manifest.

ApplicationSet

An `ApplicationSet` is a group or set of `Application` resources.

AppProject

An `AppProject` is a logical grouping that defines which Git repositories, clusters, and namespaces a set of `Applications` can access, providing multi-tenancy and security boundaries within ArgoCD.

Run the following command to list all installed CRDs. You will find the Argo CD CRDs in the command output:

```
$ kubectl get crds
```

NAME	CREATED AT
<code>applications.argoproj.io</code>	<code>2025-03-21T23:02:40Z</code>
<code>applicationsets.argoproj.io</code>	<code>2025-03-21T23:02:39Z</code>
<code>appprojects.argoproj.io</code>	<code>2025-03-21T23:02:39Z</code>
<code>argocdexports.argoproj.io</code>	<code>2025-03-21T23:02:39Z</code>
<code>argocds.argoproj.io</code>	<code>2025-03-21T23:02:39Z</code>
<code>notificationsconfigurations.argoproj.io</code>	<code>2025-03-21T23:02:39Z</code>

As with any other Kubernetes object, you can render the details. The details of the CRD will reveal the kind, API group and version, and its properties. This command inspects the `Application` CRD:

```
$ kubectl describe crd applications.argoproj.io
```

For brevity, I do not show the lengthy output. In the next section, we are going to create a CR for the `Application` CRD.

Instantiating a CR for one of the CRDs

The `Application` CRD describes a schema for defining an application representation that lives in a Git repository so that it can be deployed to a Kubernetes cluster.

For that purpose, create a new YAML manifest of kind `Application` in the file `nginx-application.yaml`, as shown in [Example 7-1](#). You may recognize some of the properties used when you rendered the CRD schema.

Example 7-1. Instantiation of CR for the `Application` CRD

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nginx
spec:
  project: default
  source:
    repoURL: https://github.com/bmuschko/cka-study-guide.git
    targetRevision: HEAD
    path: ./ch07/nginx
  destination:
    server: https://kubernetes.default.svc
    namespace: default
```

Create a new object from the YAML manifest:

```
kubectl apply -f nginx-application.yaml
application.argoproj.io/nginx created
```

Interacting with a CR

You can interact with the CR like any other object in Kubernetes. All `kubectl` create, read, update, and delete (CRUD) functions are available. For example, to list the object, use the `describe` command. The following command shows the operations in action:

```
$ kubectl describe application nginx
Name:          nginx
Namespace:     default
Labels:        <none>
Annotations:   <none>
API Version:   argoproj.io/v1alpha1
```

```
Kind:      Application
...
```

To delete the object, use the `delete` command, as shown here:

```
$ kubectl delete application nginx
application.argoproj.io "nginx" deleted
```

We demonstrated that an operator can install CRDs into the cluster, and that we can create CR from the CRDs and interact with them.

Going deeper using Argo CD would require a chapter on its own and is beyond the coverage of the certification. For more information on Argo CD, check out the book [*Argo CD: Up and Running*](#) by Andrew Block and Christian Hernandez (O'Reilly, 2025).

Inspecting the Controller

The Argo CD CRs just represent data and won't be useful by themselves. A controller acts as a reconciliation process by inspecting the state of CR objects via calls to the Kubernetes API to perform a deployment process to the cluster.

The Argo CD Operator runs the controller logic within a Pod managed by a Deployment. You can discover the Argo CD controller objects as follows:

```
$ kubectl get deployments,pods -n operators
```

NAME	READY	UP-TO-DATE
deployment.apps/argocd-operator-controller-manager	1/1	1

NAME	READY	STATUS
pod/argocd-operator-controller-manager-6998544bff-zx8bg	1/1	Running

Because we installed the operator using OLM, the controller Pod would be placed into the `operators` namespace to keep them separate from other objects in the cluster. You can scale the number of replicas as needed by changing the configuration of the Deployment.

Summary

Kubernetes refers to the CRD and the corresponding controller as the operator pattern. The Kubernetes community has implemented many operators to fulfill custom requirements. You can install them into your cluster to reuse the functionality.

A CRD schema defines the structure of a custom resource. The schema includes the group, name, version, and its configurable attributes. New objects of this kind, CRs, can be created after registering the schema. You can interact with a custom object using `kubectl` with the same CRUD commands used by any other primitive.

CRDs realize their full potential when combined with a controller implementation. The controller implementation inspects the state of specific custom objects and reacts based on their discovered state.

Exam Essentials

Know how to install operators

Operators built and managed by the Kubernetes community are available on searchable sites like Artifact Hub and OperatorHub.io. You will find installation instructions on the corresponding web pages. You do not have to memorize them for the exam. If you want to explore further, install an open source operator, such as the [Prometheus Operator](#) or the [Jaeger Operator](#).

Acquire a high-level understanding of configurable options for a CRD schema

You are not expected to implement a custom CRD schema. All you need to know is how to discover and interact with them using `kubectl`. Practice the definition of a CR in the form of a YAML manifest, and create the objects for it. Controller implementations are definitely outside the scope of the exam.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. You decide to manage a [MongoDB](#) installation in Kubernetes with the help of the [official community operator](#). This operator provides a CRD. After installing the operator, you will interact with the CRD. Navigate to the directory `app-a/ch07/mongodb-operator` of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Install the operator using the following command: `kubectl apply -f mongodbcommunity.mongodb.com_mongodbcommunity.yaml`. List all CRDs using the appropriate `kubectl` command. Can you identify the CRD that was installed by the installation procedure? Inspect the schema of the CRD. What are the type and property names of this CRD?

2. Navigate to the directory *app-a/ch07/backup-crd* of the checked-out GitHub repository [bmuschko/cka-study-guide](https://github.com/bmuschko/cka-study-guide).

Create the CRD from the file *backup-resource.yaml*. Retrieve the details for the `Backup` custom resource created in the previous step.

Create a CR named `nginx-backup` for the CRD in the `default` namespace. Provide the following property values:

- `cronExpression: 0 0 * * *`
- `podName: nginx`
- `path: /usr/local/nginx`

Retrieve the details for the `nginx-backup` object created in the previous step.